

Artificial Neural Network (ANN)



Non Symbolic Representations

- Decision trees can be easily read
 - A disjunction of conjunctions (logic)
 - We call this a symbolic representation
- Non-symbolic representations
 - More numerical in nature, more difficult to read
- Artificial Neural Networks (ANNs)
 - A Non-symbolic representation scheme
 - They embed a giant mathematical function
 - To take inputs and compute an output which is interpreted as a categorisation
 - Often shortened to “Neural Networks”
 - Don't confuse them with real neural networks (in heads)

Function Learning

- Map categorisation learning to numerical problem
 - Each category given a number
 - Or a range of real valued numbers (e.g., 0.5 - 0.9)
- Function learning examples
 - Input = 1,2,3,4 Output = 1,4,9,16
 - Here the concept to learn is squaring integers
 - Input = [1,2,3], [2,3,4], [3,4,5], [4,5,6]
 - Output = 1, 5, 11, 19
 - Here the concept is: $[a,b,c] \rightarrow a*c - b$
 - The calculation is more complicated than in the first example
- Neural networks:
 - Calculation is much more complicated in general
 - But it is still just a numerical calculation

Complicated Example: Categorising Vehicles

- Input to function: pixel data from vehicle images
 - Output: numbers: 1 for a car; 2 for a bus; 3 for a tank

INPUT



OUTPUT = 3

INPUT



OUTPUT = 2

INPUT



OUTPUT = 1

INPUT



OUTPUT=1

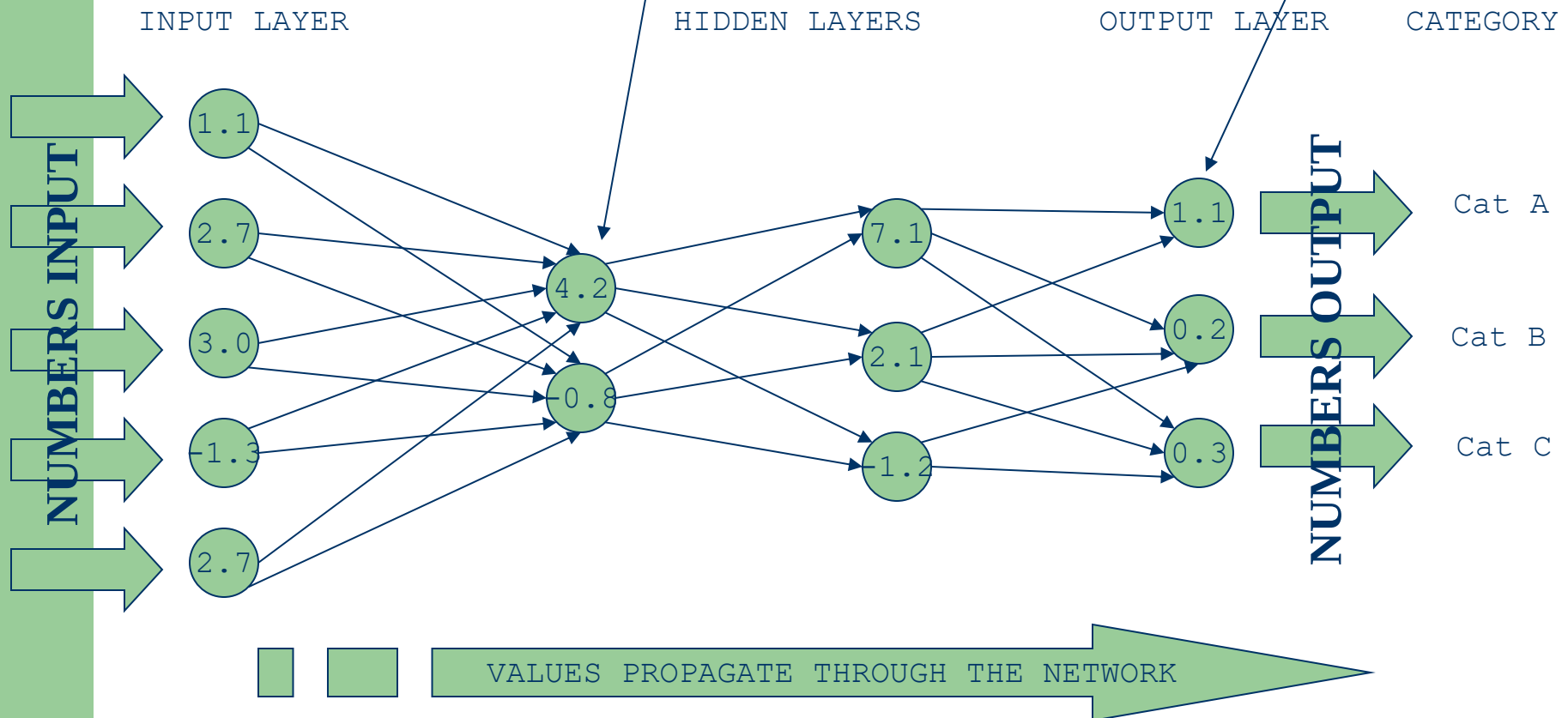
So, what functions can we use?

- Biological motivation:
 - The brain does categorisation tasks like this easily
 - The brain is made up of networks of neurons
- Naturally occurring neural networks
 - Each neuron is connected to many others
 - Input to one neuron is the output from many others
 - Neuron “fires” if a weighted sum S of inputs $>$ threshold
- Artificial neural networks
 - Similar hierarchy with neurons firing
 - Don’t take the analogy too far
 - Human brains: 100,000,000,000 neurons
 - ANNs: < 1000 usually
 - ANNs are a gross simplification of real neural networks

General Idea

Value calculated using
all the input unit values

Choose Cat A
(largest output value)



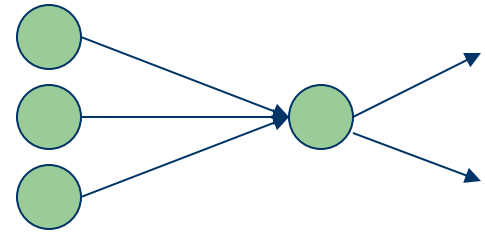
Representation of Information

- If ANNs can correctly identify vehicles
 - They then contain some notion of “car”, “bus”, etc.
- The categorisation is produced by the **units** (nodes)
 - Exactly how the input reals are turned into outputs
- But, in practice:
 - Each unit does the same calculation
 - But it is based on the **weighted sum** of inputs to the unit
 - So, the weights in the weighted sum
 - Is where the information is really stored
 - We draw weights on to the ANN diagrams (see later)
- “Black Box” representation:
 - Useful knowledge about learned concept is difficult to extract

ANN learning problem

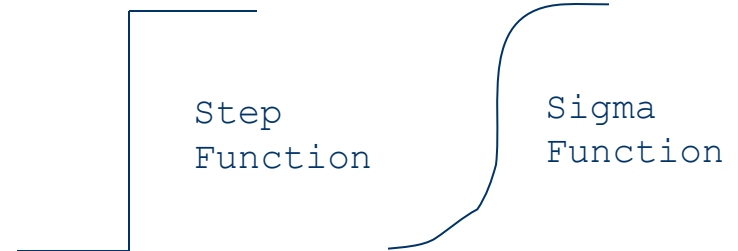
- Given a categorisation to learn (expressed numerically)
 - And training examples represented numerically
 - With the correct categorisation for each example
- Learn a neural network using the examples
 - which produces the correct output for unseen examples
- Boils down to
 - (a) Choosing the correct network architecture
 - Number of hidden layers, number of units, etc.
 - (b) Choosing (the same) function for each unit
 - (c) Training the weights between units to work correctly

Perceptrons



- Multiple input nodes
- Single output node
 - Takes a weighted sum of the inputs, call this S
 - Unit function calculates the output for the network
- Useful to study because
 - We can use perceptrons to build larger networks
- Perceptrons have limited representational abilities
 - We will look at concepts they can't learn later

Unit Functions

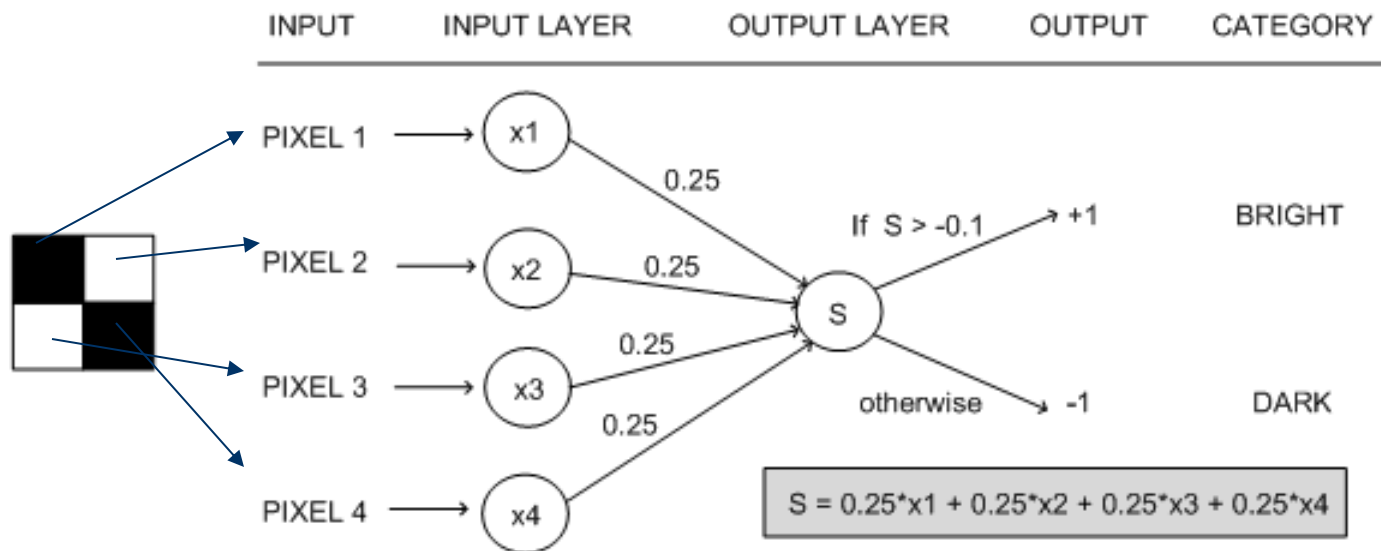


- Linear Functions
 - Simply output the weighted sum
- Threshold Functions
 - Output low values
 - Until the weighted sum gets over a threshold
 - Then output high values
 - Equivalent of “firing” of neurons
- Step function:
 - Output +1 if $S > \text{Threshold } T$
 - Output -1 otherwise
- Sigma function:
 - Similar to step function but differentiable

Example Perceptron

- Categorisation of 2x2 pixel black & white images
 - Into “bright” and “dark”
- Representation of this rule:
 - If it contains 2, 3 or 4 white pixels, it is “bright”
 - If it contains 0 or 1 white pixels, it is “dark”
- Perceptron architecture:
 - Four input units, one for each pixel
 - One output unit: +1 for white, -1 for dark

Example Perceptron

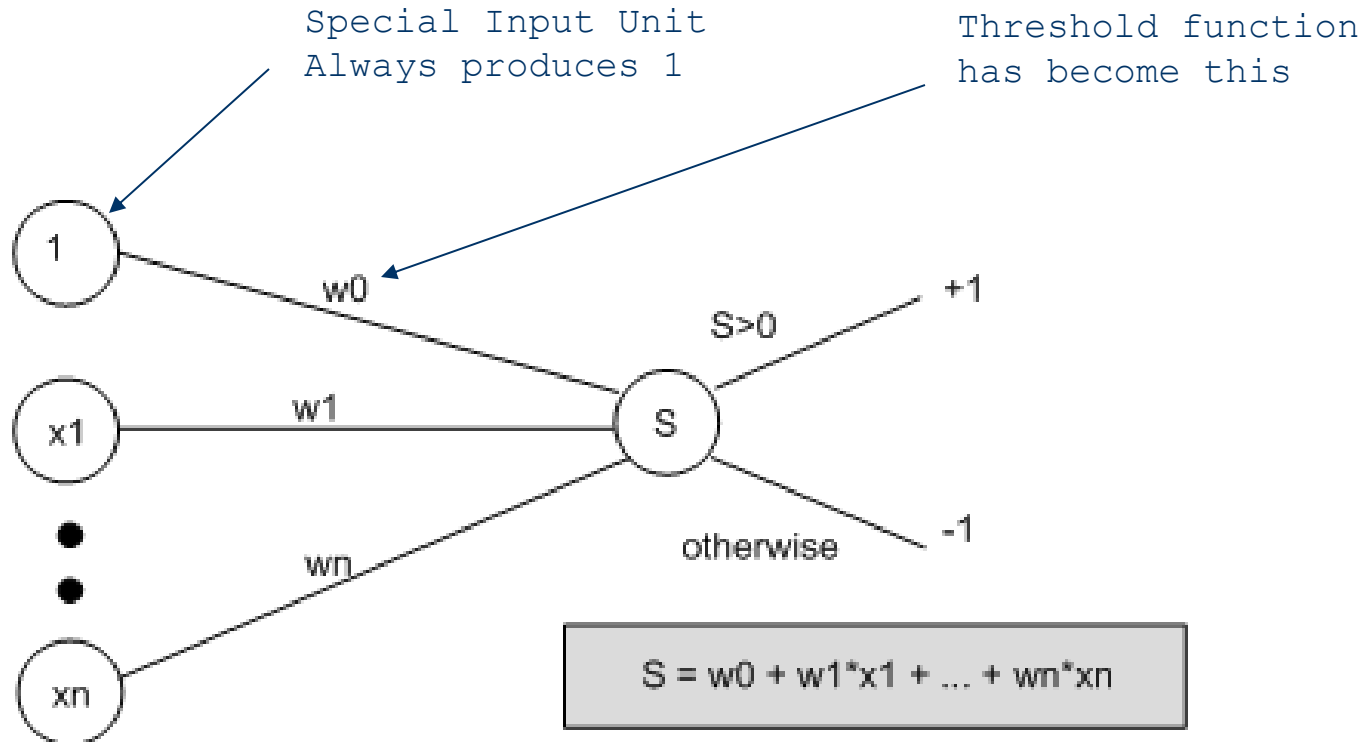


- Example calculation: $x_1 = -1$, $x_2 = 1$, $x_3 = 1$, $x_4 = -1$
 - $S = 0.25 \cdot (-1) + 0.25 \cdot (1) + 0.25 \cdot (1) + 0.25 \cdot (-1) = 0$
- $0 > -0.1$, so the output from the ANN is $+1$
 - So the image is categorised as “bright”

Learning in Perceptrons

- Need to learn
 - Both the weights between input and output units
 - And the value for the threshold
- Make calculations easier by
 - Thinking of the threshold as a weight from a special input unit where the output from the unit is always 1
- Exactly the same result
 - But we only have to worry about learning weights

New Representation for Perceptrons



Learning Algorithm

- Weights are set randomly initially
- For each training example E
 - Calculate the observed output from the ANN, $o(E)$
 - If the target output $t(E)$ is different to $o(E)$
 - Then tweak all the weights so that $o(E)$ gets closer to $t(E)$
 - Tweaking is done by perceptron training rule
- This routine is done for every example E
- Don't necessarily stop when all examples used
 - Repeat the cycle again (an 'epoch')
 - Until the ANN produces the correct output
 - For *all* the examples in the training set (or good enough)

Perceptron Training Rule

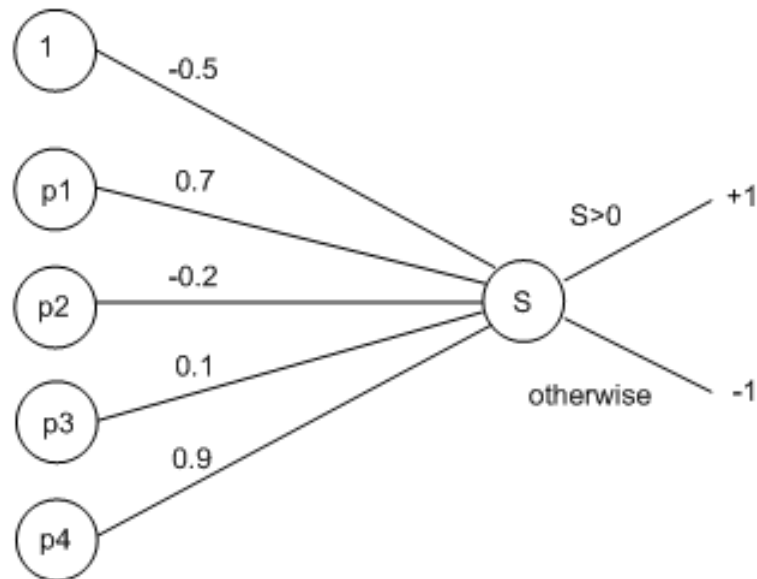
- When $t(E)$ is different to $o(E)$
 - Add on Δ_i to weight w_i
 - Where $\Delta_i = \eta(t(E) - o(E))x_i$
 - Do this for every weight in the network
- Interpretation:
 - $(t(E) - o(E))$ will either be +2 or -2 [cannot be the same sign]
 - So we can think of the addition of Δ_i as the movement of the weight in a direction
 - Which will improve the networks performance with respect to E
 - Multiplication by x_i
 - Moves it more if the input is bigger

The Learning Rate

- η is called the learning rate
 - Usually set to something small (e.g., 0.1)
- To control the movement of the weights
 - Not to move too far for one example
 - Which may over-compensate for another example
- If a large movement is actually necessary for the weights to correctly categorise E
 - This will occur over time with multiple epochs

Worked Example

- Return to the “bright” and “dark” example
- Use a learning rate of $\eta = 0.1$
- Suppose we have set random weights:



Worked Example

- Use this training example, E, to update weights:



- Here, $x_1 = -1$, $x_2 = 1$, $x_3 = 1$, $x_4 = -1$ as before
- Propagate this information through the network:
 - $S = (-0.5 * 1) + (0.7 * -1) + (-0.2 * +1) + (0.1 * +1) + (0.9 * -1) = -2.2$
- Hence the network outputs $o(E) = -1$
- But this should have been “bright”=+1
 - So $t(E) = +1$

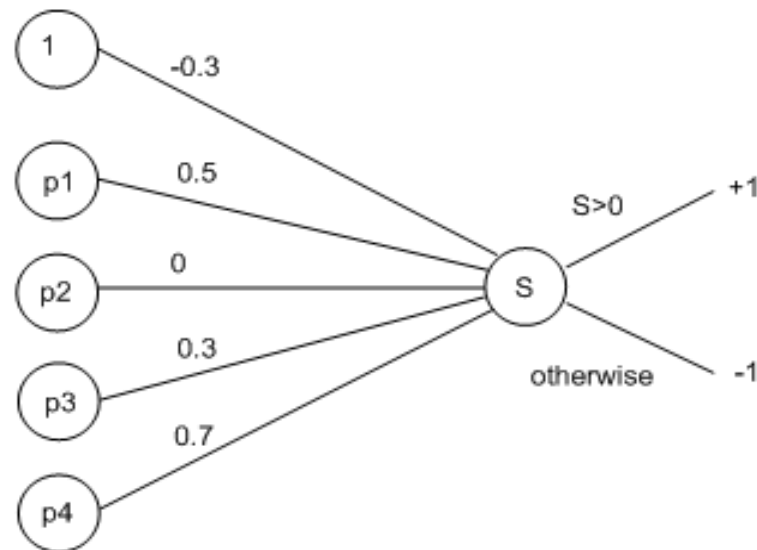
Calculating the Error Values

- $\Delta_0 = \eta(t(E)-o(E))x_0$
 $= 0.1 * (1 - (-1)) * (1) = 0.1 * (2) = 0.2$
- $\Delta_1 = \eta(t(E)-o(E))x_1$
 $= 0.1 * (1 - (-1)) * (-1) = 0.1 * (-2) = -0.2$
- $\Delta_2 = \eta(t(E)-o(E))x_2$
 $= 0.1 * (1 - (-1)) * (1) = 0.1 * (2) = 0.2$
- $\Delta_3 = \eta(t(E)-o(E))x_3$
 $= 0.1 * (1 - (-1)) * (1) = 0.1 * (2) = 0.2$
- $\Delta_4 = \eta(t(E)-o(E))x_4$
 $= 0.1 * (1 - (-1)) * (-1) = 0.1 * (-2) = -0.2$

Calculating the New Weights

- $w'_0 = -0.5 + \Delta_0 = -0.5 + 0.2 = -0.3$
- $w'_1 = 0.7 + \Delta_1 = 0.7 + -0.2 = 0.5$
- $w'_2 = -0.2 + \Delta_2 = -0.2 + 0.2 = 0$
- $w'_3 = 0.1 + \Delta_3 = 0.1 + 0.2 = 0.3$
- $w'_4 = 0.9 + \Delta_4 = 0.9 - 0.2 = 0.7$

New Look Perceptron



- Calculate for the example, E, again:
 - $S = (-0.3 * 1) + (0.5 * -1) + (0 * +1) + (0.3 * +1) + (0.7 * -1) = -1.2$
- Still gets the wrong categorisation
 - But the value is closer to zero (from -2.2 to -1.2)
 - In a few epochs time, this example will be correctly categorised